

Week 6 Lab Assignment:
Threat Modeling Your Capstone Project
Jaxon Doolittle & Keenan Johnson
GSW University

7/19/2026

App Overview

App Name: Project Aegis (from Greek mythology)

Core Concept: A highly secure, offline-first mobile password manager and digital vault for Android.

Target Users: Security-conscious individuals who want to store credentials locally without relying on vulnerable third-party cloud servers.

General Functionality & Features:

- **Biometric Gatekeeping:** The app is completely locked down and requires a master PIN or native Android fingerprint authentication to even open the dashboard.
- **Encrypted Storage:** Stores standard credential data (Account Name, URL, Username, Password) in a heavily encrypted local SQLite database.
- **Masked UI:** The main dashboard hides all passwords by default until the user explicitly toggles a visibility icon or copies them.
- **Anti-Scraping Clipboard:** When a user copies a password to paste it into another app, Project Aegis triggers a background timer that wipes the Android clipboard cache after 30 seconds to prevent malicious apps from scraping it.
- **Zero-Knowledge Sync (Optional/Stretch Goal):** If we use Firebase, we only push the fully encrypted ciphertext to the cloud, meaning the server never actually sees the plaintext passwords.

Architecture & Component List

Core Components:

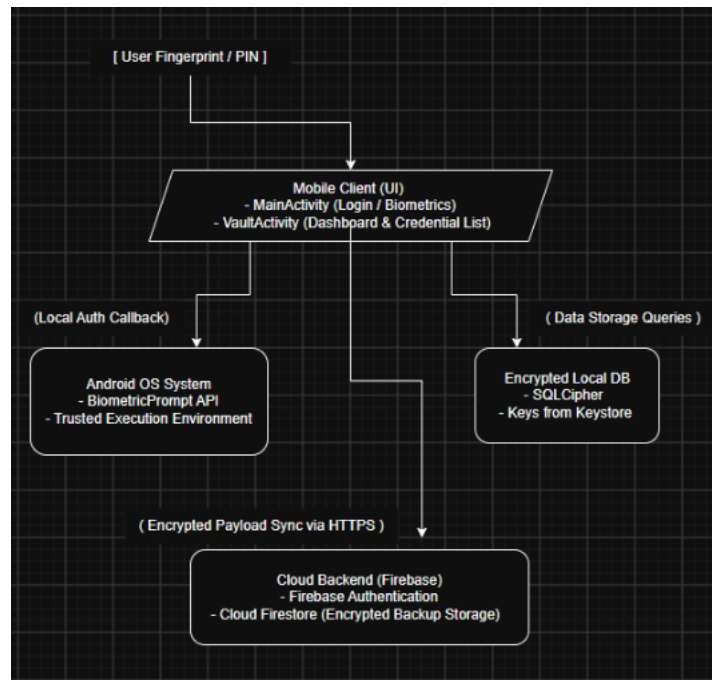
Mobile Client/Frontend UI: A native Android application written in Java

Local Storage Mechanism: Encrypted SQLCipher database system + Android Keystore system

External Android System Services: BiometricPrompt via Android's Trusted Execution Environment + Android's ClipBoardManager.

Cloud Backend/Database: FireBase, specifically using Cloud Firestore + Firebase Authentication

Architecture Diagram



Threat Modeling Table

<u>Component/ Boundary</u>	<u>STRIDE Category</u>	<u>Threat</u>	<u>Potential Impact</u>	<u>Proposed Mitigation or Defense</u>
Cloud Sync	Tampering / Information Disclosure	MitM interceptor attempts to capture or alter sync traffic passing to Firebase Cloud Firestore	Eavesdropping on communications or interception of system payloads	Enforce TLS 1.3 for all web calls; apply strict certificate pinning within the network security configuration file to block custom root certificates.
Android Clipboard	Information Disclosure	A background application monitors the Android clipboard system, scraping the raw password.	Exposure of highly sensitive plaintext credentials to unauthorized third-party apps.	Implement a custom background handler to wipe clipboard cache after a short amount of time or after pasting.
Firebase Backup Storage	Information Disclosure	Compromise of the Firebase console or cloud server infrastructure leaks the backed-up database records.	Mass exposure of stored data payloads to external actors on the public internet.	Maintain strict client-side zero-knowledge architecture. Database records are fully encrypted using client-managed AES-256 keys before transport.
Android OS Interface	Elevation of Privilege	Exploits inside older OS kernels allow processes to break isolation boundaries and read memory footprints.	Attacker extracts master keys or plaintext vault variables straight out of runtime memory.	Implement FLAG_SECURE across all UI Activities to prevent background screen scraping and OS snapshots. Clear password variables from RAM arrays immediately after usage.
Local Device Cache	Tampering	A malicious app with elevated privileges modifies the local SQLCipher file structure or configuration.	Data corruption, loss of vault accessibility, or localized denial of service.	Enforce rigid file permissions using internal application storage. Verify structural integrity parameters using SQLCipher internal health checks

Security Planning Reflection

Which parts of the app will be the most security-critical?

The most security-critical parts of the app will be the boundary between the Android Keystore and the SQLCipher local database. Because the architecture is being built with offline-first and zero-knowledge in our minds, the security of local storage keys is critical. If an attacker manages to extract the master encryption key from memory or bypasses the keystore integration, the encrypted SQLite vault becomes vulnerable to offline brute-force attempts. Additionally, the Android Clipboard interaction represents a critical interface. It is the brief moment where plaintext credentials transit outside the application's secure memory space into the OS ecosystem.

How will threat modeling affect your implementation decisions?

Threat modeling will reframe how our implementation proceeds by giving us proactive defensive guardrails to establish. Past the security risks we already planned to protect against, it revealed the need for custom handling of cache in the ClipboardManager to prevent attacks during this short period of time. It also revealed how we will handle the compromise of our third party database by protecting the data before even touching the internet. We can use our knowledge of potential security risks to establish a secure applicational foundation for our user base.